

Partiel Module d'ALGORITHMIQUE III L3I

NOTES

- ♦ Les réponses aux questions doivent être claires et justifiées.
- ♦ Pour chaque algorithme (ou fonction en C), vous aurez soin d'expliquer les différentes actions de l'algorithme ainsi que le passage des paramètres.
- ♦ **Chaque algorithme (ou fonction en C) supplémentaire utilisé doit être défini.**

EXERCICE 1 (4.25PTS)

Notes : Pour chaque équation de la complexité, vous devriez expliquer la démarche suivie et de justifier les termes qui la composent.

- 1) En ne comptabilisant que les **additions** (lignes I et II); étudier la complexité en temps de l'algorithme tita.

Procédure tita(n, s) DEBUT si n > 1 alors m ← n / 3 s ← s + m (I) tita(m, s) tita(m, s) tita(m, s) Sinon s ← s + n (II) FinSi FIN	Paramètres Formels : n : Entier (E) s : Entier (E/S) Variables intermédiaires : m : Entier
---	--

- 2) En ne comptabilisant que les **comparaisons** entre les éléments du tableau (ligne I); étudier la complexité en temps de l'algorithme tri ci-dessous :

Procédure tri(n, tab) Debut Pour i variant de n à 2 faire Pour j variant de 1 à i-1 faire Si tab[j] > tab[j+1] alors (I) tmp ← tab[j] tab[j] ← tab[j+1] tab[j+1] ← tmp Fin Si Fin Pour afficheTab(n, tab) // Résultat intermédiaire Fin Pour Fin	Paramètres Formels : n : Entier(E) tab : tableau de TElement (E/S) Variables intermédiaires : tmp : TElement
--	--

- 3) Donnez les résultats successives produites par l'appel de la procédure afficheTab lors de l'appel de la procédure tri(n, tab) avec n=5 et tab=[7, 5, 3, 4, 9].

EXERCICE 2 (3.25PTS)

- En utilisant le principe de la **sentinelle**; décrire le principe de la fusion de deux tableaux triés donnés.
- En utilisant le principe de la **sentinelle**; Ecrire en langage algorithmique ou en C, un algorithme **récuratif** nommé « fusionSentinelle » qui effectue la fusion de deux tableaux triés donnés.
- Proposer en C**, un programme qui teste l'algorithme « fusionSentinelle ».

EXERCICE 3 (2.5 PTS)

- En utilisant les primitives de la structure Pile; écrire en langage algorithmique ou en C, un algorithme nommé « invEntier » qui calcule l'inverse d'un entier donné (supposé positif).
- Proposer en C**, un programme qui teste l'algorithme « invEntier ».

Note : on suppose connu, les opérations de la structure pile

PROBLÈME (10.0 PTS)ES

Le but de ce problème est de trier par ordre croissant une liste chaînée donnée. Pour réaliser ce tri, on procède en deux étapes comme suit :

ETAPE 1 : création d'une multi-Listes à partir d'une liste chaînée :

La liste chaînée initiale sera décomposée en plusieurs sous liste triées par ordre croissant (une sous liste est aussi une liste chaînée).

Chaque sous liste sera constituée des éléments consécutifs croissants à partir de la liste initiale.

Les sous listes ainsi obtenues seront stockées dans un tableau de listes et qu'on nommera dans ce qui suit une multi-Listes et dont la structure est définie comme suit :

```
typedef int TElement;
typedef struct cellule{
    TElement donnee ;
    struct cellule *suivant ;
} *Liste ;
```

```
typedef struct{
    int nblisteMax ; // nombre maximum de sous listes
    int nbListe ; //nombre de sous listes existantes dans une MultiListes
    Liste *tab ; // tableau de sous listes
}MultiListes;
```

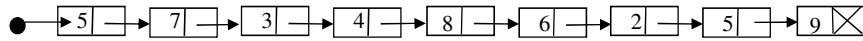
ETAPE 2 : Passage d'une multi-Listes en une liste chaînée triée par ordre croissant :

Ce passage sera effectué par fusion successives des différents sous listes qui composent la multi-Listes comme suit : on parcourt la multi-Listes donnée et on fusionne les deux premières sous listes de la multi-Listes et la sous liste fusion ainsi obtenue sera fusionnée avec la troisième sous liste de la multi-Listes et ainsi de suite...

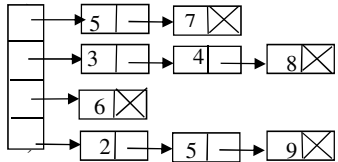
L'exemple ci-dessous illustre le tri d'une liste chaînée en utilisant la structure intermédiaire

multi-Listes :

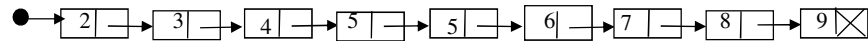
a) Liste chaînée initiale qu'on souhaite triée



b) Création de la structure intermédiaire multi-Listes à partir de la liste chaînée initiale.



c) Liste chaînée triée obtenue par fusion successive des différents sous listes de la multi-Listes.



- 1) **Ecrire en C**, l'allocation mémoire d'une multi-Listes en fonction d'un nombre donné « nMax » qui représente le nombre maximum de sous listes.
- 2) Ecrire en langage algorithmique ou en C ; l'algorithme de transformation d'une liste chaînée donnée, en une multi-Listes.
- 3) Ecrire en langage algorithmique ou en C ; **un algorithme itératif** qui déterminer l'élément de la multi-Listes qui possède le plus d'éléments.
- 4) **Ecrire en C ; un algorithme récursif** qui déterminer l'élément de la multi-Listes qui possède le plus d'éléments.
- 5) Ecrire en langage algorithmique ou en C ; l'algorithme de transformation d'une multi-Listes donnée en une liste chaînée triée par ordre croissant.
- 6) **Ecrire en C**, la libération mémoire occupée par une multi-Listes donnée.
- 7) **Ecrire en C**, un algorithme qui effectue le tri d'une liste donnée.

*****Annexe*****

On suppose connu les algorithmes suivants :

Fonction donnée : Liste $\rightarrow$ TElement

Fonction suivant : Liste $\rightarrow$ Liste

Fonction initL : [] $\rightarrow$ Liste

Fonction estVideL : Liste $\rightarrow$ Booléenne

Fonction inserTete : TElement x Liste $\rightarrow$ Liste

Fonction inserQueue : TElement x Liste $\rightarrow$ Liste

Fonction suppTete : Liste $\rightarrow$ Liste

.....

Fonction nbListe : MultiListes $\rightarrow$ Entier

Fonction nbListeMax : MultiListes $\rightarrow$ Entier

Fonction IEmeEltsML : Entier x MultiListes $\rightarrow$ Liste

Fonction estVideML : MultiListes $\rightarrow$ Booléenne

Fonction estPleinML : MultiListes $\rightarrow$ Booléenne